Building with Foundation Models

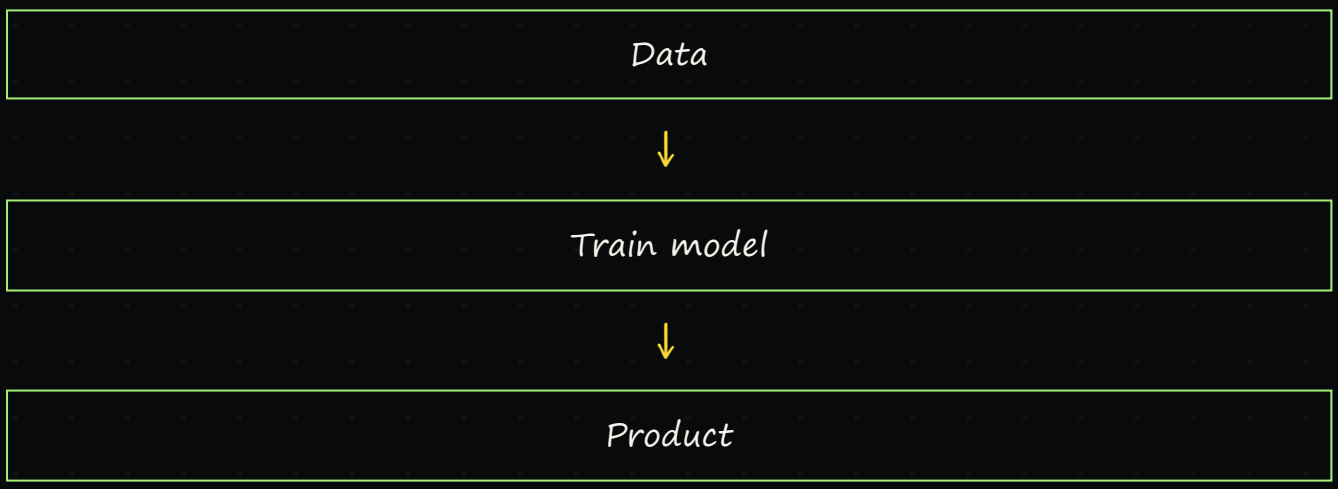
How AI engineering differs from traditional ML engineering

AI engineering focuses on adapting foundation models into useful products. The work spans prompts, context, data, evaluation, interfaces, deployment, and iteration with users.

01 AI Engineering vs. ML Engineering

Traditional ML engineering

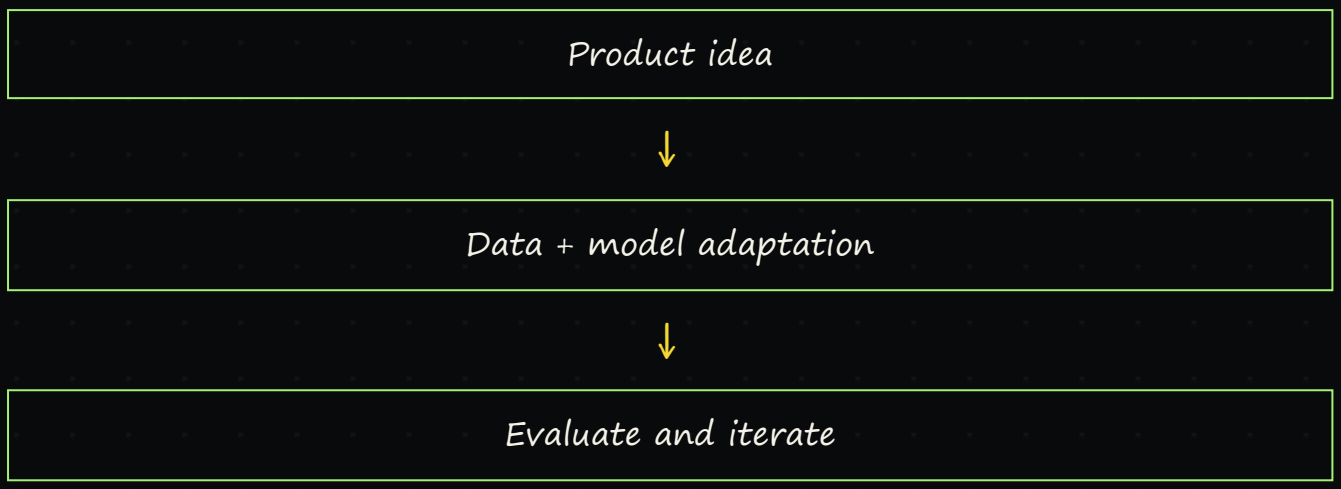
Often centers on building, training, and deploying a task-specific model. The typical flow is:



The team may focus primarily on the data-to-model portion. For conventional classification, outputs are usually drawn from a fixed set of labels, making ground-truth evaluation comparatively direct.

AI engineering

Usually starts from an already-trained foundation model and adapts it to an application rather than training a frontier-scale model from scratch.



Because output is often open-ended, evaluation is harder. The engineer is closer to the full product loop: user need, data, model behavior, interface, and deployment.

Why foundation models changed the workflow

- General-purpose capabilities grew as models, compute, and training data scaled.
- Model-as-a-service APIs let developers use powerful models without hosting large GPU clusters themselves.
- Lower infrastructure and coding barriers expanded who can prototype AI applications.
- Foundation models can support many tasks through prompting, retrieval, and fine-tuning.
- Production work still requires product judgment, data governance, evaluation, and reliability engineering.

02 Language Models and Self-Supervision

Language model (LM)

A language model captures statistical patterns in one or more languages and learns to predict tokens. A **token** is the basic unit processed by the model; it may be a word, part of a word, punctuation, or another text unit.

Tokenization maps raw text into token IDs. Subword methods, including byte-pair encoding, use a finite vocabulary while allowing unfamiliar words to be represented as smaller known pieces.

Why self-supervised learning scales

Instead of manually labeling every training example, the data itself supplies prediction targets. For next-token prediction, earlier tokens are the input and the next token is the target.

> <BOS> I love → predict "good"
> <BOS> I love good → predict the next token

This creates many learning examples from ordinary text and reduces the manual-labeling bottleneck. It enabled training on very large token collections.

Two language-model objectives

- **Masked language model:** predict a hidden token using context on both sides. BERT is a familiar example.
- **Autoregressive language model:** predict the next token using only preceding tokens; future tokens are not visible. This is the common text-generation setup.

Pre-training teaches broad patterns through token prediction. Post-training then teaches the model to follow instructions and exhibit desired behaviors.

Parameters

Parameters are learned numerical values that encode patterns useful to the model. Training updates parameters to improve the chosen objective; prompting does not itself update them.

03 From LMs to Foundation Models

Broader capabilities

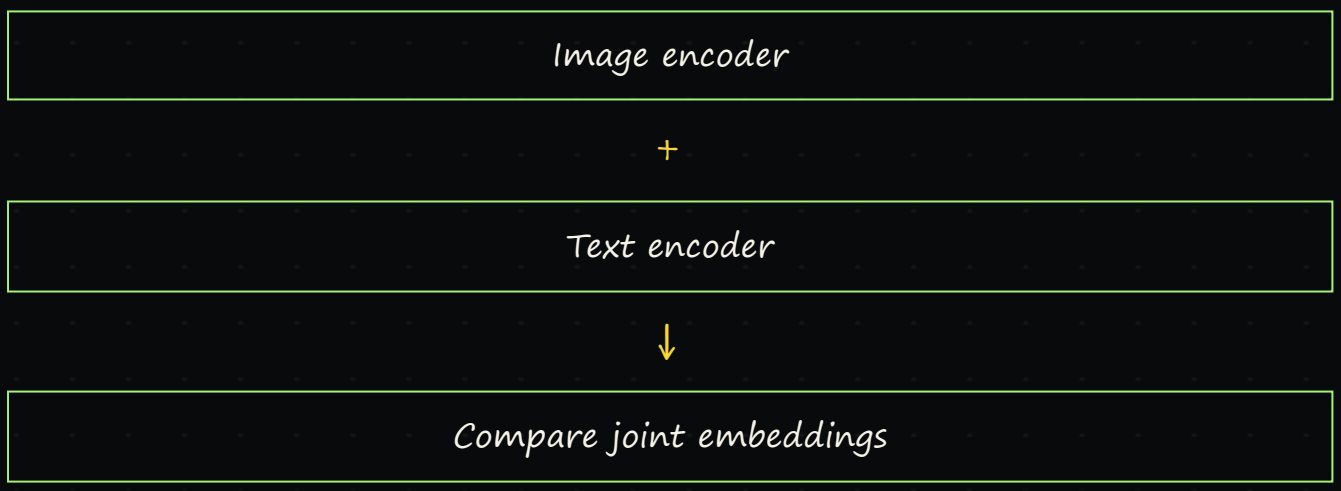
A foundation model is trained broadly and can be adapted to many downstream tasks. Language-only models process text; multimodal foundation models can represent or process combinations such as text and images, and may also work with audio or video.

In a multimodal model, a prediction can depend on multiple modalities, not only preceding text tokens.

Embeddings and CLIP

An **embedding** is a vector representation intended to capture useful meaning or relationships in data.

CLIP (Contrastive Language-Image Pre-training) learns a shared representation for paired images and captions. Matching image-text pairs are encouraged to be close in a joint embedding space; unrelated pairs are separated.



Open-ended generation

Generative models can produce many valid outputs for one request. A summary, explanation, or conversation does not have one simple ground-truth label, so the quality criteria must account for relevance, factuality, coherence, safety, and usefulness.

04 Three Ways to Adapt a Foundation Model

1. Prompting

Specify the task, constraints, desired format, and relevant examples in the input. Prompt engineering improves the instructions and context sent to the model without changing its weights.

2. Retrieval-augmented generation (RAG)

Retrieve relevant information from a data source and place it in the prompt. Grounding answers in current or private information can improve relevance and reduce unsupported claims, though retrieval quality and context design still matter.

3. Fine-tuning

Train on examples to change model behavior. Parameter-efficient methods update a smaller subset of parameters; full fine-tuning updates the full model. Choose based on task, data, compute, and measured benefit.

Common application areas

Conversation, writing, coding, image or video production, education, information synthesis across workplace sources, and workflow automation are examples. Enterprise use often places additional emphasis on privacy, access control, and risk.

05 Plan the Product Before Scaling the Model

Use-case evaluation

Start with a real user or business problem. Ask whether an AI solution is worthwhile relative to its cost, effort, operational risk, and expected benefit. Competitive pressure alone does not establish that a use case will succeed.

Keep a human in the loop where oversight, correction, or approval is important. Product development is iterative: prototype, evaluate, learn from use, and revise.

Set measurable expectations

Translate business outcomes into AI-system measures instead of treating model scores as the goal.

- **Quality:** task success, factuality, relevance, and safety.
- **Latency:** time to first token (TTFT), time per output token (TPOT), and total response time.
- **Cost:** cost per request, token usage, and serving compute.
- **Responsible use:** fairness, privacy, and interpretability where relevant.

> Approximate total generation time = TTFT + (TPOT × generated output tokens)

Milestones and maintenance

Define milestones, delivery dates, and explicit success criteria. After launch, monitor quality, usage, cost, and risks; maintain the system and revisit its cost-benefit case as user behavior and business needs change.

AI engineering is a product loop



Foundation-model APIs let teams test a product idea without first building a foundation model. Real usage can reveal which data and model improvements are worth investing in.

06 The AI Engineering Stack

Application development

- Evaluation of the user-facing behavior.
- Prompt engineering and context construction.
- Retrieval, tools, and agent workflows.
- User interfaces and product experience.

Model development

- Modeling, training, and adaptation.
- Dataset engineering and curation.
- Model evaluation.
- Inference optimization for speed and cost.

Infrastructure

- Compute and data management.
- Serving models reliably.
- Monitoring, logging, and operational support.

Key distinction

ML engineering often owns a narrower data-to-model pipeline. AI engineering connects product design to foundation-model adaptation, application behavior, and operations. It is broader and more full-stack, while still relying on collaboration across specialties.

Prepared from the supplied introductory transcript. Technical examples are summarized in their lecture context.